Technical Documentation

RAG-Based Customer Support Assistant

1. Introduction

🔹 What is RAG

Retrieval-Augmented Generation (RAG) is an approach where a system retrieves relevant information from a knowledge base and uses it to generate accurate, context-aware responses.

🔹 Why RAG is Needed

Traditional LLM-based systems often:

- Hallucinate (generate incorrect answers)
- Lack domain-specific knowledge

RAG solves this by:

- Grounding responses in real data
- Improving reliability and accuracy

🔹 Use Case Overview

This project implements a **Customer Support Assistant** that:

- Answers FAQs from a PDF knowledge base
- Provides accurate, context-based responses
- Escalates to human when needed

2. System Architecture Explanation

The system consists of the following components:

1. **Document Ingestion Pipeline**
    - Loads PDF
    - Splits into Q&A chunks
    - Generates embeddings
2. **Vector Database (ChromaDB)**
    - Stores embeddings

    - Enables semantic search
3. **Retriever**
    - Fetches relevant chunks based on query
4. **LLM (Groq)**
    - Generates answers using retrieved context
5. **LangGraph Workflow**
    - Controls execution flow
    - Routes output or escalation
6. **HITL Module**
    - Handles fallback when no answer is found

3. Design Decisions

🔹 Chunk Size Strategy

- Instead of fixed-size chunks, **Q&A-based chunking** is used
- Ensures each chunk contains complete information
- Prevents retrieval of incomplete answers

🔹 Embedding Strategy

- HuggingFace embeddings (all-MiniLM-L6-v2)
- Lightweight and efficient
- Suitable for semantic similarity

🔹 Retrieval Approach

- Vector similarity search using ChromaDB
- Top-k retrieval for relevant context

🔹 Prompt Design Logic

- Strict grounding:
    - Answer only from context
    - No extra information

- Balanced:
    - Avoid under-answering
    - Avoid hallucination

4. Workflow Explanation (LangGraph)

🔹 **Nodes**

1. Process Node

- Retrieves context
- Generates answer

2. Decision Node

- Evaluates:
    - Context availability
    - Answer validity

3. Output Node

- Returns final response

4. HITL Node

- Handles escalation

🔹 **State Transition**

```
Input → Process → Decision
                  ├── Output
                  └── HITL
```

5. Conditional Logic

🔹 **Intent Detection**

- Based on context availability and answer validity

- **Routing Decisions**

```
if context is empty:
    escalate
elif answer contains "I don't know":
    escalate
else:
    return answer
```

- **Key Principle**
  - Prefer **no answer over wrong answer**

6. HITL Implementation

- **Role of Human Intervention**
  - Ensures reliability when system cannot answer

- **When Triggered**
  - No relevant context
  - Unknown query
  - Low confidence

- **Behavior**
  - Returns fallback message:

```
Escalated to human agent. Please wait for support.
```

- **Benefits**
  - Prevents misinformation
  - Improves trust

- **Limitations**
  - Requires manual handling

- Not fully automated

7. Challenges & Trade-offs

🔹 **Retrieval Accuracy vs Speed**

- More chunks → better accuracy but slower
- Balanced using optimized chunking

🔹 **Chunk Size vs Context Quality**

- Small chunks → better precision
- Large chunks → more context but noisy

Solution:

- Q&A-based chunking

🔹 **Cost vs Performance**

- Larger models → better quality but higher cost
- Groq provides fast and efficient inference

8. Testing Strategy

🔹 **Approach**

- Test with:
    - Valid queries
    - Edge cases
    - Out-of-context queries

🔹 **Sample Queries**

✓ Valid:

- What is your return policy?
- How can I reset my password?

✓ Edge:

- refund???

✓ Invalid:

- Explain machine learning

🔹 **Expected Behavior**

- Valid → correct answer
- Invalid → HITL

9. Future Enhancements

- Multi-document support
- Confidence scoring instead of rule-based routing
- Reranking for better retrieval
- Memory integration
- UI (Streamlit / Web app)
- Deployment (cloud-based API)